src/config.py

283 lines

```
"""
Central configuration for the Autonomous Production Choke Controller project.

EVERY tunable number in the project lives here so that the plant, the operating
envelope and the controller can be re-tuned without touching any logic.

Units used throughout the project
---------------------------------
Flow rate Q : bbl/hr (barrels per hour)
Pressures WHP/FLP/BHP : psi
Choke opening u : % (0 = fully shut, 100 = fully open)
Time : hours (control interval Ts = 1 h)
"""

from __future__ import annotations

from dataclasses import dataclass, field, asdict
from pathlib import Path

# ---------------------------------------------------------------------------
# Paths
# ---------------------------------------------------------------------------
ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
FIG_DIR = ROOT / "figures"
REPORT_DIR = ROOT / "report"
DASH_DIR = ROOT / "dashboard"

for _d in (DATA_DIR, FIG_DIR, REPORT_DIR, DASH_DIR):
_d.mkdir(exist_ok=True)

# ---------------------------------------------------------------------------
# Global reproducibility
# ---------------------------------------------------------------------------
SEED = 20260725 # fixed master seed -> every result is reproducible
TS_HOURS = 1.0 # control interval Ts, per the problem statement


# ---------------------------------------------------------------------------
# Plant (simulator) parameters
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class PlantParams:
"""
Physics parameters of the naturally flowing well stand-in simulator.

The numbers below were chosen to give field-realistic magnitudes for a
modest naturally flowing oil well:

* reservoir pressure ~3000 psi at ~5000 ft TVD
* productivity index typical of a moderately productive sandstone
* flowing rates of order 100-170 bbl/hr (2,400-4,100 bbl/day)
* wellhead pressures of a few hundred psi while flowing
* flowline / separator backpressure of order 200 psi
"""

# ---- Reservoir inflow performance (linear IPR) ------------------------
# BHP = P_res - Q / PI (drawdown grows linearly with rate)
p_res: float = 3000.0 # psi average reservoir pressure
pi_index: float = 0.15 # bbl/hr per psi productivity index

# ---- Production tubing ------------------------------------------------
# WHP = BHP - dp_static - k_fric * Q^2
# Static head: 5000 ft TVD x 0.30 psi/ft oil gradient = 1500 psi
tvd_ft: float = 5000.0 # ft true vertical depth
grad_psi_per_ft: float = 0.30 # psi/ft fluid gradient (live oil)
# Friction is turbulent -> grows with Q^2. k_fric is sized so that the
```

```python
# friction loss is ~50 psi at 150 bbl/hr, which is typical for this size
# of tubing / rate.
k_fric: float = 50.0 / 150.0 ** 2 # psi / (bbl/hr)^2

# ---- Production choke (orifice equation) ----------------------------
# Q = Cv * f(u) * sqrt(WHP - FLP)
# Flow depends on BOTH the opening and the pressure drop across the valve.
# f(u) = (u/100)^choke_exp gives the characteristic "slow to open, then
# rapidly effective" behaviour of a real production choke.
cv: float = 30.0 # bbl/hr / (psi^0.5) valve capacity
choke_exp: float = 1.5 # valve characteristic exponent

# ---- Flowline / gathering system ------------------------------------
# FLP = flp_base + k_flowline * Q
flp_base: float = 180.0 # psi separator + manifold backpressure
k_flowline: float = 0.40 # psi per bbl/hr flowline friction

# ---- First-order lag time constants (hours) -------------------------
# Nothing in the well moves instantly. BHP is the slowest (reservoir /
# near-wellbore storage), the flowline is the fastest.
tau_q: float = 1.0
tau_whp: float = 1.2
tau_flp: float = 0.8
tau_bhp: float = 2.0

# ---- Sensor noise (1 sigma, Gaussian) -------------------------------
noise_q: float = 0.8 # bbl/hr
noise_whp: float = 1.5 # psi
noise_flp: float = 1.0 # psi
noise_bhp: float = 2.0 # psi

# ---- Informational outputs (not active constraints) -----------------
# Wellhead temperature rises with rate (less heat loss at higher rate);
# annulus pressure is essentially static for a naturally flowing well.
wht_base: float = 95.0 # degF at zero flow (ambient-ish)
wht_gain: float = 0.35 # degF per bbl/hr
tau_wht: float = 3.0 # hours (thermal mass -> slow)
annulus_pressure: float = 450.0 # psi, nominally constant
noise_wht: float = 0.3
noise_ap: float = 1.0


PLANT = PlantParams()


# ---------------------------------------------------------------------------
# Safe operating envelope (active constraints)
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class Limits:
"""
Active operating constraints. The controller must never allow the well
to leave this envelope.

Where the numbers come from
---------------------------
BHP_min = 1900 psi : maximum allowable drawdown (sand-control / bubble
point protection). With the linear IPR this caps
the *maximum safe flow rate* at
Q_max = (3000 - 1900) * 0.15 = 165 bbl/hr
-> this is the binding constraint of the problem and
is what makes Scenario C genuinely infeasible.
WHP_min = 320 psi : minimum wellhead pressure needed to keep the well
flowing stably into the flowline. Becomes active at
~168 bbl/hr, i.e. just after BHP -> a second, nearly
active constraint.
FLP_max = 260 psi : flowline / separator design backpressure.
The max/min bookends (WHP_max, BHP_max, FLP_min) bound the shut-in state.
"""

```

```
whp_min: float = 320.0
whp_max: float = 1600.0 # shut-in WHP is ~1500 psi
flp_min: float = 150.0
flp_max: float = 260.0
bhp_min: float = 1900.0 # <-- the binding constraint
bhp_max: float = 3050.0 # shut-in BHP is 3000 psi

def as_dict(self) -> dict:
return asdict(self)


LIMITS = Limits()


# ---------------------------------------------------------------------------
# Controller configuration
# ---------------------------------------------------------------------------
@dataclass(frozen=True)
class ControllerParams:
"""Tuning of the brute-force MPC."""

ts: float = TS_HOURS # control interval (h)

# Choke actuator constraints (from the problem statement)
u_min: float = 0.0
u_max: float = 100.0
max_move: float = 5.0 # +/- % per control interval (ramp-rate limit)
candidate_step: float = 0.5 # brute-force grid resolution -> 21 candidates

# Prediction horizon [steps]. This MUST outrun the slowest process lag:
# BHP has tau = 1.76 h, so a 5-step horizon only sees ~94 % of the final
# pressure drop and the controller happily parks at a "safe" point that
# then keeps sinking after the horizon ends. The Monte-Carlo study caught
# exactly that (11 of 100 Scenario-C runs breached BHP). 10 steps = 10 h
# = 5.7 x tau_BHP, so predictions are settled to within 0.3 %.
horizon: int = 10

# Objective: J = (Q_pred_end - Q_target)^2 + move_penalty * du^2
move_penalty: float = 0.05

# Minimum cost improvement (in (bbl/hr)^2) that a move must buy before the
# controller will move at all. Without this the argmin of a noisy cost
# function jitters every interval and the choke chatters at the constraint
# boundary - real valve wear for no production benefit. 2.0 corresponds to
# roughly 1.4 bbl/hr of predicted improvement.
min_cost_improvement: float = 2.0

# Safety back-off applied *inside* each hard limit so that the true plant
# can never cross the line. The margin has to cover THREE error sources,
# not just noise:
# 1. sensor noise (~2 psi 1-sigma on BHP)
# 2. plant/model mismatch (piecewise-linear steady-state curve)
# 3. measurement-filter lag (the filtered value trails the true one
# while the well is still moving)
# Sizing on noise alone (10 psi on BHP) let a 0.1 psi breach through once
# the measurement filter was added, so the margins below are set with
# headroom and verified over 100 random seeds by
# scripts/06_robustness_study.py.
margin_whp: float = 10.0
margin_flp: float = 8.0
margin_bhp: float = 15.0

# Measurement filtering for the disturbance (bias) estimator
bias_filter_alpha: float = 0.3

# First-order filter on the measurements the predictor starts from.
# Raw sensor noise moves the predicted trajectories across the constraint
# boundary at random, which makes the *feasible set* itself flap and the
# choke hunt when the well is pinned against a limit. Filtering costs a
# little response lag (~1.5 steps at alpha=0.4) but is what makes the
```

```python
# constrained steady state actually steady. Raw measurements are still
# what the safety audit checks.
meas_filter_alpha: float = 0.4


CONTROLLER = ControllerParams()


# ---------------------------------------------------------------------------
# Step-test experiment design
# ---------------------------------------------------------------------------
STEP_HOLD_HOURS = 8 # ~4x the slowest time constant -> true steady state

# Identification experiment: up AND down steps, small and large, spanning the
# whole 0-100 % range, with 5 % spacing through 30-70 % where the well
# actually operates and where the steady-state curves are most strongly
# convex.
#
# Why the spacing matters: the model interpolates the steady-state curve
# linearly between knots, and for a convex curve the chord lies ABOVE the
# truth - i.e. the model over-estimates BHP, which is optimistic in exactly
# the wrong direction for a safety constraint. The original design jumped
# 55 -> 70 % and the resulting ~8 psi chord error at the binding operating
# point put 11 of 100 Monte-Carlo runs over the BHP limit. Halving the knot
# spacing cuts that interpolation error by ~9x (it scales with spacing^2).
STEP_SEQUENCE_ID = [0, 20, 25, 30, 40, 35, 45, 55, 50, 60, 65, 70, 80, 90, 100]

# Validation experiment (held out from identification): deliberately chosen at
# choke positions that are NOT identification knots, so the validation really
# does test interpolation rather than replaying fitted points.
STEP_SEQUENCE_VAL = [0, 25, 63, 42, 85]


# ---------------------------------------------------------------------------
# Demonstration scenarios
# ---------------------------------------------------------------------------
SCENARIOS = {
"A": {
"name": "Scenario A - Startup to Target",
"description": (
"The well starts shut-in (choke = 0 %, no flow). The controller "
"must bring it up to a 120 bbl/hr target safely, respecting the "
"5 %/step ramp limit and the full operating envelope."
),
"duration_h": 60,
"u_initial": 0.0,
"targets": [(0, 120.0)], # (start hour, target bbl/hr)
"expected": "Reaches 120 bbl/hr with no constraint violation.",
},
"B": {
"name": "Scenario B - Target Tracking",
"description": (
"The controller holds 100 bbl/hr, then the production target is "
"raised to 150 bbl/hr at t = 50 h. It must re-target while "
"respecting WHP/FLP/BHP limits and the choke ramp rate."
),
"duration_h": 100,
"u_initial": 0.0,
"targets": [(0, 100.0), (50, 150.0)],
"expected": "Tracks 100 then 150 bbl/hr, no constraint violation.",
},
"C": {
"name": "Scenario C - Infeasible Target",
"description": (
"A 200 bbl/hr target is requested. The maximum safe rate is "
"~165 bbl/hr (limited by BHP >= 1900 psi). The controller must "
"refuse to chase the target, settle at the maximum achievable "
"safe rate, and never breach a limit."
```

```
),
"duration_h": 100,
"u_initial": 0.0,
"targets": [(0, 200.0)],
"expected": "Settles at max safe rate (~160-165 bbl/hr), no violation.",
},
}

```